



Figure 3: Graph Output1

Visualising Graph using DOT

The DOT language describes three main objects: graph, node and edges. A DOT file starts with either 'graph' or 'digraph', followed by a graph name. 'digraph' is used for a directed graph and 'graph' for an undirected graph. The name can contain any alphanumeric character, but spaces are not allowed. Nodes of the graph are contained within curly braces. Double hyphens represent the edges of the undirected graph while the 'arrow' is used for showing directionality. The following example represents a four-node undirected graph:

```
graph hello {
  a -- b -- c;
  b -- d;
}
```

It can be turned into a directed graph by changing 'graph' to 'digraph' and replacing the double hyphen with arrows. The following is the modified script:

```
digraph graphname {
  a -> b -> c;
  b -> d;
}
```

Some of the important attributes that can be controlled in the DOT file are the node shape, labels, colour, fonts, style of connections, etc. One graph may enclose another graph and it is specified as a sub-graph in DOT. Vertices in the outer node can even share vertices with the enclosed sub-graph's node:

```
graph hello2 {
// Hello World with nice colors and big fonts

Node1 [label="Hello, World!", color=Blue, fontcolor=Red,
      fontsize=24, shape=box, style=filled]

Node2 [label="Hello, Again!", color=Red, fontcolor=Blue,
```

```
      fontsize=24, shape=oval, style=solid]
```

```
Node3 [label="Hello, Again!", color=Blue, fontcolor=Blue,
      fontsize=24, shape=circle, style=dashed]
```

```
Node1 -- Node2 [color=red]
```

```
Node1 -- Node3 [style=dotted,color=red]
```

```
subgraph hello3{
      Node4 [label="Hello, Again!", color=Blue,
      fontcolor=Blue,
      fontsize=24, shape=circle, style=dashed]
```

```
      Node5 [label="Hello, Again!", color=Blue,
      fontcolor=Blue,
      fontsize=24, shape=circle, style=dashed]
```

```
Node4 -- Node5
}
```

```
Node3 -- Node5
}
```

The DOT language only specifies the graph and the layout is taken care of by various layout engines present as part of Graphviz. In GVEdit, users can switch the layout engine easily. It is even possible to generate this DOT file programmatically and many languages provide bindings for that. The presence of the dot command line utility helps in automating the graph creation too.

Apart from those discussed in this article, there are many more powerful DSLs available for visualising data. Each library provides a different level of functionality; some provide readily available charts with a few configurations, like Vega, and some provide implementation from scratch, like Processing. A few other notable libraries for visualisation are gRaphaël JS, a good charting library that supports older browsers; Paper JS for rendering vector graphics in Canvas; Kartograph JS for building interactive maps; and Tangle JS for creating user interactive reactive documents. **END** 🐼

References

- [1] <http://processing.org/>
- [2] <http://trifecta.github.io/vega/>
- [3] <http://www.graphviz.org/>

By: Karthikeyan R and AnandaVelMurugan Chandra Mohan

Karthikeyan R is a Java developer and technophile who loves to keep in touch with emerging technologies.

Anand is a data analyst who loves playing with data using R. He likes exploring new technologies.